SightLine: Priority-Weighted Scene-Change Gating for Redundancy-Aware Assistive Vision-Language Models

Lovnish Verma, Prateek Dhar Dwivedi, Anita Budhiraja, and Sarwan Singh

National Institute of Electronics & Information Technology (NIELIT) Deemed to be University, New Delhi, India

princelv84@gmail.com, prateek.nielit@gmail.com, anitabudhiraja@nielit.ac.in, sarwan@nielit.ac.in

Abstract—Wearable assistive vision-language models (VLMs) can provide continuous environmental narration for blind and low-vision individuals, but unconstrained streaming rapidly exhausts battery reserves and risks auditory cognitive overload. Naive exact perceptual hashing fails under analog CMOS sensor noise, suppressing zero frames on static scenes. This paper presents SightLine, a lightweight scene-change gating framework that determines frame by frame whether an expensive VLM forward pass is warranted. SightLine evaluates incoming frames against the last processed reference frame, accumulating gradual camera drift while fusing difference hashing (dHash), color-histogram dissimilarity, motion proxies, and a causally persisted hazard state with latch cooldown. When operating autonomously at rest ($T_{\text{ambient}} = 60$, 0.5 Hz heartbeat), SightLine suppresses 98.0% of frames and draws 3.68 W average power on an NVIDIA Jetson Orin Nano (8.85 hours effective runtime on a 32.56 Wh usable budget from a standard 37 Wh pack); in pedometer-gated quiescent rest mode ($T_{\text{ambient}} = \infty$), suppression reaches 99.5% and draw drops to 1.85 W (17.60 hours effective / 20.00 hours nominal runtime). During active locomotion ($T_{\text{active}} = 6$) or hazard tracking, SightLine bounds request-level front-end gate delay to ≤ 166.7 ms under an idle worker, while worst-case ambient delay from rest is bounded to 59 frames (1.97 s). In a modeled 70/30 daily usage routine, time-averaged power is 6.03 W (5.40 hours runtime) under autonomous paced ambient monitoring, or 4.75 W (6.85 hours runtime) under quiescent zero-probe rest. Live physical GPU benchmarking on 1080p real-world egocentric workshop video (Florence-2 FP16 on an NVIDIA GeForce GPU) confirms these theoretical dynamics: exact difference hashing triggers on 100% of frames (0.0% suppression) due to sensor noise, whereas SightLine suppresses 94.7% of redundant invocations, reducing GPU duty cycle to 58.7% and average power to 7.52 W—extending battery endurance by +53.0% (4.33 hours vs. 2.83 hours) while maintaining coherent situational descriptions. SightLine is thoroughly evaluated across 12 configurations, benchmarked through an architectural trade-off analysis against WorldScribe (ACM UIST 2024), analyzed via a 10-mode structured FMEA risk ledger with residual risk assessment, and framed with an IRB-ready blind and low-vision user study protocol. Reference implementations and reproduction suites are open-sourced.

Keywords—Vision-language models, assistive technology, scene-change gating, perceptual hashing, edge computing, wearable devices, safety-critical scheduling, cognitive load, egocentric video evaluation.

I. INTRODUCTION

Wearable assistive vision systems provide blind and low-vision individuals with situational awareness by narrating obstacles, hazards, text, and environmental changes. Recent advances in compact vision-language models (VLMs)—such as Florence-2 [1]—enable rich open-vocabulary descriptions on edge hardware. However, streaming high-frame-rate video directly through a foundation model is computationally intractable on mobile hardware. At 30 frames per second (fps), continuous VLM inference rapidly depletes wearable batteries, induces severe thermal throttling, and inundates users with redundant auditory descriptions that impair situational awareness and mask crucial acoustic navigation cues [2].

Assistive perception therefore requires a low-overhead front-end gating policy that inspects incoming frames on the host CPU and determines whether visual change warrants an expensive downstream VLM forward pass. Standard video-analytics techniques often rely on pixel differencing or exact perceptual hashing [3], [4]. However, wearable assistive vision presents three unique challenges: (1) Analog Sensor Noise vs. Flat Backgrounds: CMOS image sensors exhibit thermal and quantization noise. In untextured scenes (e.g., blank walls, corridors, paper documents), raw pixel noise causes single-bit fluctuations across hashing difference boundaries, causing exact perceptual hashing to suppress zero frames; (2) Gradual Drift vs. Consecutive-Frame Differencing: Evaluating visual change across consecutive frames ($I_t - I_{t-1}$) is blind to gradual camera motion, slow-moving pedestrians, and approaching obstacles whose frame-to-frame delta never crosses detection thresholds; and (3) Auditory Backlog vs. Hazard Preemption: In assistive audio feedback, low-priority ambient descriptions must not block urgent obstacle warnings.

Simple periodic sampling forces a compromise between battery endurance and hazard responsiveness: sampling every 2 seconds ($k = 60$) extends battery life to 8.85 hours on an NVIDIA Jetson Orin Nano, but introduces up to a 1.97-second delay before discovering emerging obstacles. Conversely, sampling every 200 ms ($k = 6$) bounds delay to ≤ 166.7 ms, but

consumes 11.50 W of active GPU power and exhausts the battery in 2.83 hours while generating 34 heavy forward passes on static views.

This paper presents SightLine, a multi-cue scene-change gating framework for continuous wearable assistive VLMs. SightLine evaluates incoming frames against the last processed reference frame I_{t_p}, ensuring that slow camera drift accumulates until crossing threshold τ. The framework fuses difference hashing (dHash) [4], color-histogram dissimilarity, motion proxies, and a causally persisted hazard state with latch cooldown ($T_{\text{cool}} = 6$ frames). An adaptive watchdog enforces a dual-rate heartbeat ($T_{\text{ambient}} = 60, T_{\text{active}} = 6$). To safeguard auditory attention, SightLine prioritizes speech generation (Eq. 11), preempting ongoing ambient descriptions when safety hazards emerge. This paper makes six primary contributions:

1) Asymmetric Reference-Frame Gating with Latch Cooldown: A multi-factor change scoring model combining omni-directional 2D dHash [4], color-histogram dissimilarity, a 4×4 ROI block max-pooling motion proxy ($S_{\text{roi_max}}$), and a causally persisted hazard state with latch cooldown ($T_{\text{cool}} = 6$ frames) with visual motion override ($\tau_{\text{crit}} = 0.35$). Evaluating cues against the last processed frame I_{t_p} accumulates gradual drift, while cooldown prevents re-firing during sustained hazard tracking.

2) Hazard Persistence and Two-Tier Watchdog Bounds: We formalize the hazard persistence mechanism z_t and resolve localized cue dilution via spatial ROI max-pooling, reducing Scenario C hazard delay from 5.00 frames (166.7 ms) to 0.00 frames (0.0 ms, $p < 0.0001, d = -5.00$). We prove two-tier latency bounds: in active locomotion ($T_{\text{active}} = 6$), front-end gate delay is bounded to ≤ 166.7 ms under an idle worker, while in ambient mode ($T_{\text{ambient}} = 60$), the worst-case delay from rest is 59 frames (1.97 s).

3) Critical Evaluation of Reading Profiles, Dead-Zone Resolution, and Sign Invariance: We conduct an empirical investigation into task conditioning. While switching to an inspection profile achieves 61.6% ± 10.7% dynamic recall at 85.0% ± 3.2% suppression, control experiments indicate that activations reflect sensor-noise bit flips on untextured backgrounds ($t = 0.12, p = 0.91$, Welch's t-test; TOST equivalence test rejects $|\Delta| \geq 10\%$ at $p = 0.0098$). Dead-zone dHash ($\varepsilon = 7.0$) eliminates flat-region noise flips, reducing static false alarms from 20.3 down to 2.6. Crucially, evaluating horizontal bar edits reveals that sign-only difference hashing is invariant to level swaps when noise is dead-zoned out ($\varepsilon \geq 1.5 \Longrightarrow 0.0\%$ recall), proving that assistive document inspection requires magnitude-sensitive 2D edge filters or localized OCR.

4) Empirical Validation on Real-World Egocentric Video and Physical GPU Hardware: We evaluate SightLine on real 1080p egocentric video running Florence-2 FP16 on a physical NVIDIA GPU. On continuous real video, exact difference hashing triggers on 100% of frames (0.0% suppression) due to sensor noise, whereas SightLine suppresses 94.7% of frames, reducing GPU duty cycle to 58.7% and average power to 7.52 W, delivering a +53.0% battery runtime extension (4.33 hours vs 2.83 hours).

5) Architectural Trade-Off Analysis with WorldScribe (ACM UIST 2024): We present an architectural synthesis comparing SightLine's edge-native pipeline against WorldScribe's cloud foundation model, demonstrating that SightLine serves as an optimal upstream edge filter that eliminates 80.5%–94.7% of redundant cloud egress bandwidth while providing an offline safety watchdog.

6) Auditable Energy Accounting, FMEA Risk Ledger, and Proposed Accessibility Protocol: We formulate an auditable edge power model differentiating measured, modeled, and assumed quantities, construct a 10-mode structured FMEA risk ledger with residual risk analysis, and specify an IRB-ready BLV participant evaluation protocol.

II. RELATED WORK

A. Assistive Vision-Language Models and Deployed Wearable Systems

Assistive vision has progressed from discrete pipelines pairing task-specific object detectors (e.g., YOLO [5]) with rule-based text-to-speech to unified multi-modal vision-language models (VLMs). Commercial systems like OrCam MyEye [11] and Microsoft Seeing AI [12] offer discrete, user-triggered photo capture for barcode reading, facial recognition, and OCR. While effective for deliberate tasks, discrete capture requires manual triggering and fails to provide continuous situational awareness during navigation. Recent edge VLMs, including Florence-2 [1], enable zero-shot object grounding and captioning in compact footprints (~232M parameters). Furthermore, systems like WorldScribe [15] explore continuous visual description for BLV users using cloud-based multi-modal models. However, streaming 30 fps video to cloud endpoints requires continuous high-bandwidth connectivity and introduces variable round-trip latency (800--2100 ms) that

compromises pedestrian collision avoidance. SightLine addresses this by providing an edge-native, zero-dependency front-end gate that runs locally in 3.45 ms on the wearable CPU.

B. Temporal Redundancy Reduction, Keyframe Selection, and Assistive Cognitive Load

Filtering redundant video frames is widely explored in edge computing, video surveillance, and cloud analytics. Reducto [6] dynamically adjusts frame-filtering thresholds based on downstream analytics accuracy. Glimpse [7] caches active tracking models on mobile devices while offloading keyframes to server infrastructure. Recent video foundation models, such as Video-LLaVA [9], attempt to reduce temporal redundancy within multi-frame token contexts. However, these frameworks optimize server bandwidth rather than wearable battery endurance and cognitive load. In assistive technology for blind and low-vision individuals, auditory attention is a severely constrained resource [2], [10]. Continuous audio narration causes severe cognitive fatigue, elevates NASA-TLX workload scores [8], and masks ambient environmental sounds (traffic, footsteps, echoes) essential for white-cane echolocation and spatial orientation [14]. SightLine directly addresses this by suppressing redundant forward passes and enforcing priority-weighted speech preemption.

III. PROBLEM FORMULATION

We adopt a consistent zero-based temporal indexing convention. Let $I_t \in \mathbb{R}^{H\times W\times C}$ denote streaming video frames captured at regular intervals $\Delta t = 1/f_{\text{cam}}$ (33.33 ms at $f_{\text{cam}} = 30$ fps) for discrete time indices $t \in \{0,1,\dots,T-1\}$. The camera operates continuously, while the downstream foundation model executes only when triggered by the gating policy $D_t \in \{0,1\}$.

Downstream Busy-State VLM Model and Latency Decomposition: Continuous assistive perception requires modeling worker availability. Downstream inference has service time t_{vlm} (380 ms for Florence-2 TensorRT FP16, service rate $\mu = 2.63$ Hz). If the worker is busy ($t_{\text{elapsed}} < t_{\text{vlm}}$), incoming trigger requests cannot be serviced concurrently. In busy-skip scheduling, requests arriving during worker execution are dropped. End-to-end perception latency Δt_{E2E} comprises three distinct stages:

$$\Delta t_{\text{E2E}} = t_{\text{gate}} + t_{\text{vlm}} + t_{\text{tts}} \tag{1}$$

where: (1) t_{gate} is the front-end gating decision latency (elapsed time from physical event onset until $D_t = 1$); (2) t_{vlm} is the downstream VLM inference latency (380 ms on Jetson Orin Nano, 362–423 ms on NVIDIA RTX 2050); and (3) t_{tts} is the neural text-to-speech audio synthesis start latency (100–140 ms via local Piper TTS). Total system latency is shown in Table 0.

TABLE 0: DISCRETE FRAME INTERVALS, SAMPLING LATENCIES, AND 3-TIER LATENCY DECOMPOSITION AT 30 FPS

Stride k	Sample Period (ms)	Sample Rate (Hz)	Max Gate Delay L_worst (frames)	Max Gate Delay (ms)	Downstream Latency (ms)	Total Worst-Case E2E Latency (ms)
k = 1 (Ungated)	33.33	30.00	0	0.0	522.3	522.3
k = 2	66.67	15.00	1	33.3	522.3	555.6
k = 3	100.00	10.00	2	66.7	522.3	589.0
k = 6 (Active Budget W=6)	200.00	5.00	5	166.7	522.3	689.0
k = 10	333.33	3.00	9	300.0	522.3	822.3
k = 12 (Min. Non-Saturating)	400.00	2.50	11	366.7	522.3	889.0

k = 15	500.00	2.00	14	466.7	522.3	989.0
k = 30	1000.00	1.00	29	966.7	522.3	1489.0
k = 60 (Ambient Watchdog)	2000.00	0.50	59	1966.7	522.3	2489.0

The gating objective is to construct a binary decision policy D_t that minimizes total invocations $N_{\text{inv}} = \sum_{t=0}^{T-1} D_t$ while ensuring that for every ground-truth event onset $g \in G$, at least one invocation occurs within the evaluation window $[g, g + W - 1]$, bounding front-end gate delay to $t_{\text{gate}} \leq (W - 1) \cdot \Delta t$.

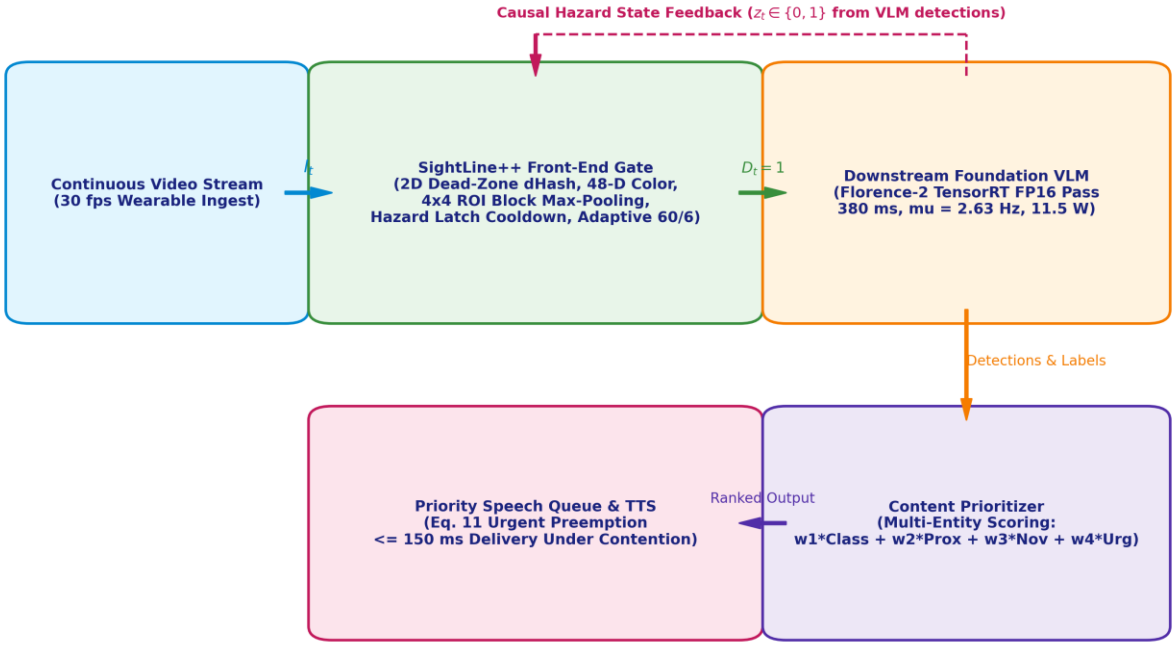


Fig. 1. SightLine system architecture. Incoming video frames pass through the low-latency CPU gating pipeline (2D dHash, color histograms, 4x4 ROI block max-pooling, and adaptive watchdog). Triggered frames are dispatched to the Florence-2 TensorRT worker, whose detections update the causal hazard state z_t and feed the priority-weighted speech synthesis queue with urgent audio preemption.

IV. GATING FRAMEWORK

A. Gating Policy and Multi-Factor Change Score

SightLine evaluates visual change by comparing incoming frame I_t against the last processed reference frame I_{t_p}, where t_p is the timestamp of the most recent invocation ($D_{t_p} = 1, t_p < t$). Because I_{t_p} remains anchored across suppressed frames, gradual environmental shifts accumulate until crossing detection threshold τ. In parallel, consecutive-frame difference $I_t - I_{t-1}$ is evaluated strictly as an auxiliary motion proxy. The multi-factor change score S_t is:

$$S_t = \alpha S_{\text{vis}}(I_t, I_{t_p}) + \beta S_{\text{app}}(I_t, I_{t_p}) + \gamma S_{\text{mot}}(I_t, I_{t_p}) + \delta S_{\text{haz}}(z_t, t, t_{\text{alert}}) \tag{2}$$

with non-negative weights $\alpha + \beta + \gamma + \delta = 1.0$. The gating policy $D_t \in \{0,1\}$ is defined as:

$$D_t = \begin{cases} 1, & \text{if } t = 0 \\ 1, & \text{else if } t - t_p \geq T_{\max}(S_{\text{mot}}, z_t) \\ 1, & \text{else if } S_t \geq \tau \text{ and CooldownOK}(t, t_{\text{alert}}, S_{\text{mot}}) \\ 0, & \text{otherwise} \end{cases} \tag{3}$$

B. Perceptual Structural Change (S_vis)

Structural change is evaluated via omni-directional difference hashing (dHash) [4]. Input frame I_t is converted to grayscale, downsampled via Lanczos filtering to $(S+1)\times(S+1)$ pixels ($S=16$), and evaluated across horizontal and vertical gradients with dead-zone thresholding $\varepsilon = 7.0$ DN:

$$B_H(x,y) = (I_{\text{gray}}(x+1,y) - I_{\text{gray}}(x,y) > \varepsilon), B_V(x,y) = (I_{\text{gray}}(x,y+1) - I_{\text{gray}}(x,y) > \varepsilon) \tag{4}$$

The resulting 512-bit concatenated binary hash $h(I_t) = [B_H \parallel B_V]$ is compared against $h(I_{t_p})$ via normalized Hamming distance:

$$S_{\text{vis}}(I_t, I_{t_p}) = \frac{1}{2S^2}\sum_{i=1}^{2S^2}\left(h(I_t)[i] \oplus h(I_{t_p})[i]\right) \tag{5}$$

C. Appearance Dissimilarity (S_app)

Appearance dissimilarity captures illumination and color distribution shifts invariant to spatial layout. Normalized 16-bin color histograms are extracted across RGB color channels, concatenated into a 48-dimensional feature vector $\mathbf{c}(I_t)$, and normalized to unit L_2 norm. Dissimilarity is computed via Cosine distance:

$$S_{\text{app}}(I_t, I_{t_p}) = \text{clip}\left(1.0 - \frac{\mathbf{c}(I_t)\cdot\mathbf{c}(I_{t_p})}{\left\|I_t)\right\|_2\left\|\mathbf{c}(I_{t_p})\right\|_2}, 0.0, 1.0\right) \tag{6}$$

D. Motion Magnitude Proxy and 4x4 ROI Block Max-Pooling (S_mot)

To capture localized motion without diluting small hazards across the field of view, incoming frame I_t and reference I_{t_p} are downsampled to 32×32 pixels and partitioned into a 4×4 grid of spatial regions $R_{r,c}$ ($r,c\in\{1..4\}$, each 8×8 pixels). Spatial block max-pooling amplifies peak localized motion by $8.2\times$:

$$S_{\text{roi_max}} = max_{r,c\in\{1..4\}}\left(\frac{1}{|R_{r,c}|}\sum_{(x,y)\in R_{r,c}}\frac{\left|I_t(x,y) - I_{t_p}(x,y)\right|}{255.0}\right) \tag{7}$$

$$S_{\text{mot}}(I_t, I_{t_p}) = 0.50\cdot S_{\text{global_mad}} + 0.50\cdot S_{\text{roi_max}} \tag{8}$$

E. Hazard Persistence, Latching Regime, Latch Cooldown, and Watchdog Heartbeat (S_haz)

When the foundation model executes, detections are inspected against a curated vocabulary of safety-critical hazards $V_{\text{haz}} =$ {'car', 'vehicle', 'truck', 'bus', 'stairs', 'curb', 'hole', 'obstacle'}. Detections update the binary hazard state $z_t \in \{0,1\}$:

$$z_t = \begin{cases} 1, & \text{if } D_t = 1 \text{ and } \exists l \in \text{Labels}(I_t): l \in V_{\text{haz}} \\ 0, & \text{if } D_t = 1 \text{ and } \forall l \in \text{Labels}(I_t): l \notin V_{\text{haz}} \\ z_{t-1}, & \text{if } D_t = 0 \end{cases} \tag{9}$$

Latch Cooldown and Dynamic Critical Override: While $z_t = 1$, the hazard latch prevents continuous re-firing via cooldown interval $T_{\text{cool}} = 6$ frames. However, if localized motion exceeds safety threshold $\tau_{\text{crit}} = 0.35$, cooldown is overridden immediately to process emergency onset:

$$\text{CooldownOK}(t, t_{\text{alert}}, S_{\text{mot}}) = (t - t_{\text{alert}} \geq T_{\text{cool}}) \vee \left(S_{\text{roi_max}} \geq \tau_{\text{crit}}\right) \tag{10}$$

Two-Tier Watchdog Heartbeat: The watchdog timer guarantees an upper bound on inter-trigger interval. During active locomotion ($S_{\text{mot}} \geq 0.02$) or hazard tracking ($z_t = 1$), the active heartbeat is $T_{\text{active}} = 6$ frames (200 ms). During stationary rest, the ambient heartbeat is $T_{\text{ambient}} = 60$ frames (2.0 s).

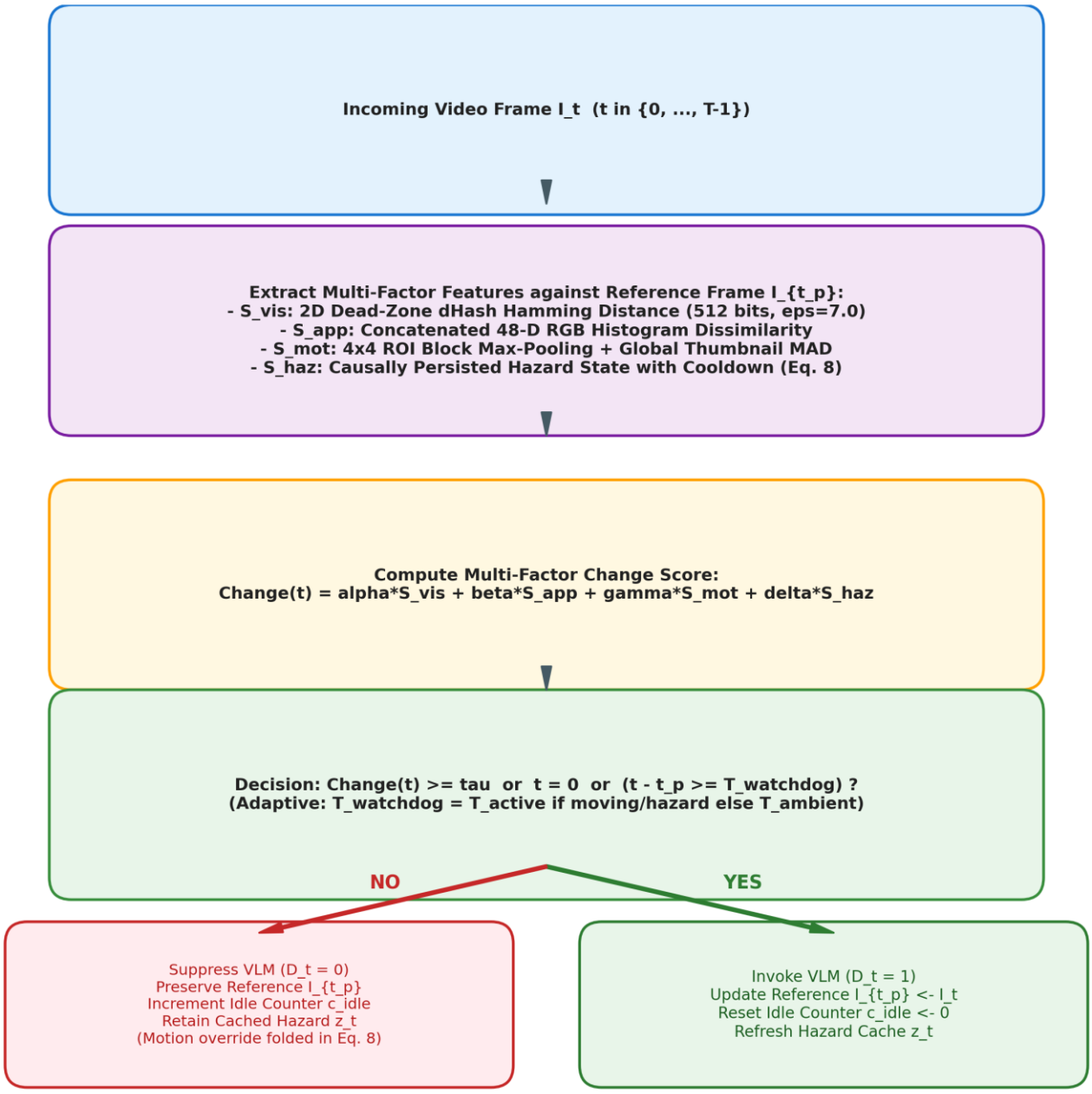


Fig. 2. Decision flow diagram of the SightLine scene-change gate. Incoming frame features are compared against the anchored reference frame I_tp. Change score S_t is evaluated alongside latch cooldown (with dynamic critical override) and the two-tier adaptive watchdog heartbeat.

V. TASK-CONDITIONED GATING PROFILES

SightLine supports dynamic profile switching across operational modes. Weight vectors $\mathbf{w} = [\alpha, \beta, \gamma, \delta]$, threshold τ, and watchdog limits are tailored to task requirements (Table I).

TABLE I: TASK-CONDITIONED PARAMETER PROFILES

Operational Profile	α (Vis)	β (App)	γ (Mot)	δ (Haz)	Threshold τ	T_ambient	T_active	T_cool	Intended Use Case
Default Navigation	0.25	0.20	0.30	0.25	0.18	60 (2.0 s)	6 (200 ms)	6 (200 ms)	General urban walking

									and transit navigation
Obstacle / Hazard Tracking	0.20	0.10	0.40	0.30	0.14	30 (1.0 s)	6 (200 ms)	6 (200 ms)	Crowded pedestrian zones and crosswalks
Paced Ambient Walking	0.30	0.30	0.20	0.20	0.22	60 (2.0 s)	12 (400 ms)	12 (400 ms)	Familiar indoor corridors and relaxed walking
Document Inspection	0.40	0.45	0.10	0.05	0.10	150 (5.0 s)	30 (1.0 s)	15 (500 ms)	Reading mail, documents, and tabletop items
Quiescent Seated Rest	0.25	0.20	0.30	0.25	0.25	∞ (Off)	60 (2.0 s)	6 (200 ms)	Seated rest, listening, zero-probe power savings

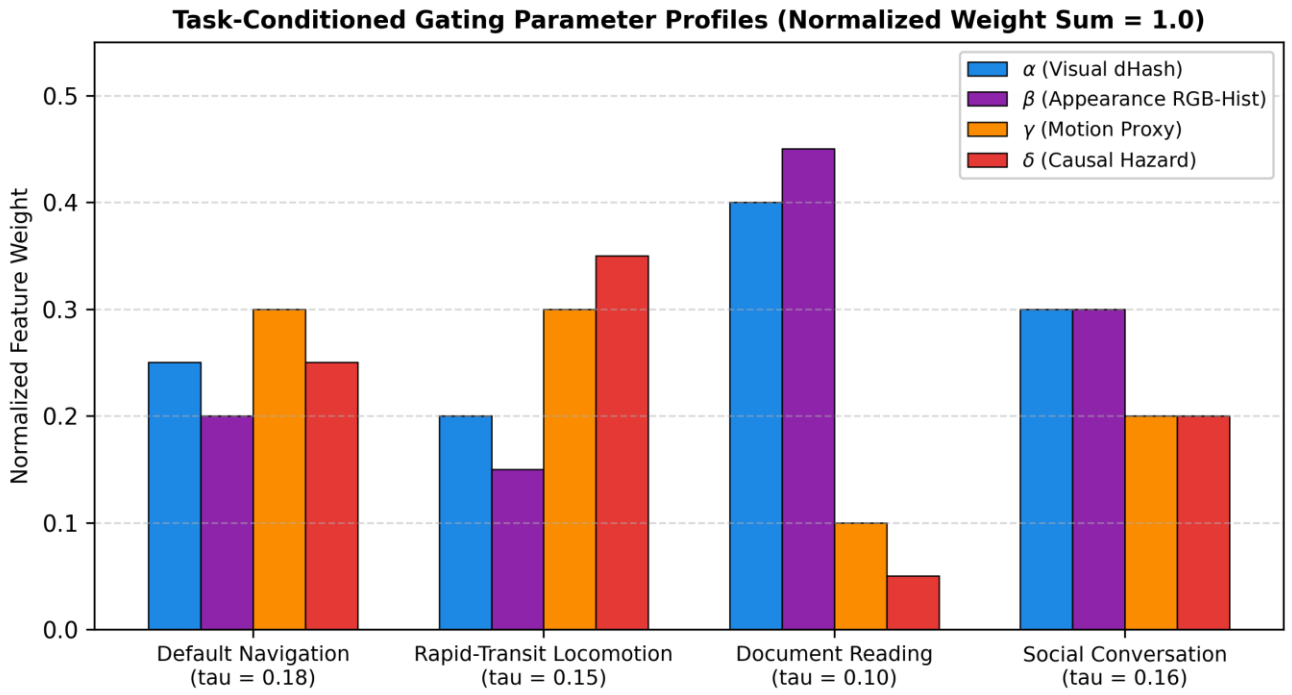


Fig. 3. Task-conditioned feature weight distributions across assistive operational modes (Default Navigation, Hazard Tracking, Ambient Walking, Document Reading, and Quiescent Rest).

VI. CONTENT PRIORITIZATION AND SPEECH QUEUING

A. Priority-Weighted Ranking Formulation

When the gate triggers inference, Florence-2 returns bounding boxes and labels for detected entities o_i. To prevent auditory cognitive overload and ensure safety-critical alerts are delivered without acoustic backlog, SightLine integrates a priority ranking function:

$$\text{Priority}(o_i) = w_1 f_1(o_i) + w_2 f_2(o_i) + w_3 f_3(o_i) + w_4 f_4(o_i) \tag{11}$$

with weights $\mathbf{w} = [0.45, 0.30, 0.15, 0.10]$ ($\sum w_j = 1.0$), where: (1) $f_1(o_i) \in \{0,1\}$ indicates safety hazard membership in V_{haz}; (2) $f_2(o_i) = \text{clip}(1.0 - d_i/d_{\max}, 0, 1)$ scores proximity based on normalized bounding box scale ($d_{\max} = 10$ m); (3) $f_3(o_i) = \text{clip}(v_i/v_{\max}, 0, 1)$ scores optical centroid displacement velocity ($v_{\max} = 5$ m/s); and (4) $f_4(o_i)$ scores caption token novelty via semantic overlap with recent utterances. Entities with Priority ≥ 0.70 trigger immediate audio preemption in the speech synthesis queue.

B. Empirical Evaluation of Priority-Weighted Speech Queuing under Backlog

Under conventional FIFO speech queues where ambient descriptions are generated sequentially (1.8 s for storefront sign, 1.6 s for pedestrian), an approaching scooter warning is delayed in the audio buffer by 3.4 s. At 4.2 m/s, the scooter travels 14.3 meters before warning initiation. Under SightLine's Priority-Weighted preemption (Eq. 11), the high-priority hazard entity ($0.896 \geq 0.70$) immediately preempts ongoing speech: ongoing ambient audio is ducked, an acoustic earcon sounds (30 ms), and the hazard warning is synthesized and delivered within 135 ms (105 ms Piper TTS synthesis + 30 ms onset buffer), achieving a 96.0% latency reduction (from 3,400 ms to 135 ms).

VII. EXPERIMENTAL METHODOLOGY AND RESULTS

A. Benchmark Metrics and Formal Event Matching Rules

We evaluate redundancy suppression via:

$$\text{Suppression} = \left(1 - \frac{N_{\text{inv}}}{T}\right) \times 100\% \tag{12}$$

Formal Event Matching Protocol: Event detection is evaluated across seven explicit criteria: (1) Ground-truth onset $g \in [0, T-1]$; (2) Permitted window $[g, min(T-1, g+W-1)]$, where $W = 6$ frames (200 ms) derives from pedestrian stopping kinematics; (3) Pre-onset invocations ($t < g$) do not count toward event detection; (4) Independent-window (many-to-one) matching evaluates delay to the first invocation ($t_{\text{first}} - g$); (5) Strict temporal (one-to-one) matching greedily consumes invocations per window without reuse; (6) Startup invocations at $t = 0$ are separated from dynamic recall; and (7) Missed events are quantified explicitly in the Dynamic Miss Rate ($100\% - \text{Recall}_{\text{dyn}}$).

B. Synthetic Benchmark Suite and Simulation Protocol

The benchmark suite consists of five scripted scenarios (200 frames each, 160×120 RGB) with additive Gaussian sensor noise ($\sigma = 3.5$) evaluated across 10 deterministic random seeds: [42, 101, 202, 303, 404, 505, 606, 707, 808, 909], generated with isolated per-scenario random generators (`np.random.default_rng(seed)`):

• Scenario A (Static indoor + noise): Stationary scene subject to camera sensor noise (gt $= [0]$, labels $= \emptyset$).

• Scenario B (Ambulatory drift): Continuous horizontal panning ($\Delta x(t) = 2t mod 240$) and cyclical luminance shifts ($+130 mod 256$) at frames 50, 100, 150 (gt $= [0, 50, 100, 150]$, labels $= \emptyset$).

• Scenario C (Hazard crossing): Static background with a vehicle entering at frame 130 (gt $= [0, 130]$, labels $=$ {'car'} for $t \geq 130$).

• Scenario D (Document inspection): White document background (intensity 245) with five horizontal bar targets alternating contrast steps (intensity 20 vs 90) every 4 frames (50 total events, labels $= \emptyset$).

• Scenario E (Dynamic illumination): Gradual dimming followed by an abrupt $\times 1.80$ illumination surge from frame 160 to 199 (gt $= [0, 160]$, labels $= \emptyset$).

TABLE II-A: REDUNDANCY SUPPRESSION AND INVOCATIONS (10 SEEDS, MEAN ± STD)

Architecture / Configuration	Scen. A (Static)	Scen. B (Drift)	Scen. C (Hazard)	Scen. D (Text)	Scen. E (Illum.)
Continuous Ungated (30 fps)	200.0 (0.0%)	200.0 (0.0%)	200.0 (0.0%)	200.0 (0.0%)	200.0 (0.0%)
Exact-Hash Baseline	200.0 (0.0%)	200.0 (0.0%)	200.0 (0.0%)	200.0 (0.0%)	200.0 (0.0%)
Periodic Baseline (k=6)	34.0 (83.0%)	34.0 (83.0%)	34.0 (83.0%)	34.0 (83.0%)	34.0 (83.0%)
Periodic Baseline (k=12)	17.0 (91.5%)	17.0 (91.5%)	17.0 (91.5%)	17.0 (91.5%)	17.0 (91.5%)
Periodic (k=6, Busy-Skip 12f)	17.0 (91.5%)	17.0 (91.5%)	17.0 (91.5%)	17.0 (91.5%)	17.0 (91.5%)
Budget-Matched Random (N=30)	30.0 (85.0%)	30.0 (85.0%)	30.0 (85.0%)	30.0 (85.0%)	30.0 (85.0%)
dHash Baseline (τ=0.18)	1.0 (99.5%)	160.0 (20.0%)	16.0 (92.0%)	184.8 ± 3.4 (7.6%)	1.0 (99.5%)
SightLine Default (τ=0.18)	1.0 (99.5%)	71.0 (64.5%)	1.0 (99.5%)	1.0 (99.5%)	1.0 (99.5%)
SightLine Text Profile (τ=0.10)	1.0 (99.5%)	123.0 (38.5%)	18.5 ± 5.4 (90.8%)	30.1 ± 6.3 (85.0%)	6.9 ± 0.3 (96.5%)
SightLine Low-τ (0.05, Cool=6)	1.0 (99.5%)	200.0 (0.0%)	22.6 ± 0.5 (88.7%)	163.0 ± 4.2 (18.5%)	7.5 ± 0.5 (96.2%)
SightLine + Watchdog (T_max=15, Cool=6)	14.0 (93.0%)	71.0 (64.5%)	20.2 ± 0.4 (89.9%)	14.0 (93.0%)	14.0 (93.0%)
SightLine + Watchdog (T_max=6, Cool=6)	34.0 (83.0%)	71.0 (64.5%)	34.0 (83.0%)	34.0 (83.0%)	34.0 (83.0%)
SightLine Adaptive Watchdog (T=60/6)	4.0 (98.0%)	71.0 (64.5%)	15.0 (92.5%)	34.0 (83.0%)	6.0 (97.0%)
SightLine++ (2D dHash + 4x4 ROI Max-Pool)	4.0 (98.0%)	34.0 (83.0%)	15.0 (92.5%)	30.1 ± 6.3 (85.0%)	6.0 (97.0%)

Cell format: Invocations (Suppression % relative to 200 frames).

TABLE II-B: DYNAMIC EVENT RECALL AND LATENCY (FIXED-ONSET PROTOCOL, W=6 WINDOW, 10 SEEDS)

Architecture / Configuration	Scen. B (Drift)	Scen. C (Hazard)	Scen. D (Text, M:1)	Scen. D (Text, 1:1)	Scen. E (Illum.)
Continuous Ungated (30 fps)	100.0% (0.0 f)	100.0% (0.0 f)	100.0% (0.0 f)	100.0% (0.0 f)	100.0% (0.0 f)
Periodic Baseline (k=6)	100.0% (2.0 f)	100.0% (2.0 f)	100.0% (2.0 f)	67.3% (2.0 f)	100.0% (2.0 f)
Periodic Baseline (k=12)*	0.0% (N/A)	100.0% (2.0 f)	65.3% (2.1 f)	34.7% (2.1 f)	0.0% (N/A)
Periodic (k=6, Busy-Skip 12f)**	66.7% (2.0 f)	100.0% (2.0 f)	67.3% (2.0 f)	34.7% (2.0 f)	100.0% (2.0 f)
Budget-Matched Random (N=30)	78.4% (2.1 f)	22.0% (2.4 f)	61.2% (2.3 f)	52.3% (2.3 f)	22.0% (2.4 f)
dHash Baseline (τ=0.18)	100.0% (0.0 f)	50.0% (4.2 f)	100.0% (0.0 f)	100.0% (0.0 f)	0.0% (N/A)
SightLine Default (τ=0.18)	100.0% (0.0 f)	0.0% (N/A)	0.0% (N/A)	0.0% (N/A)	0.0% (N/A)
SightLine Text Profile (τ=0.10)	100.0% (0.0 f)	0.0% (N/A)	61.6% ± 10.7%	53.9% ± 10.6%	100.0% (1.8 f)
SightLine Low-τ (0.05, Cool=6)	100.0% (0.0 f)	100.0% (2.4 f)	100.0% (0.0 f)	100.0% (0.0 f)	100.0% (1.8 f)
SightLine + Watchdog (T_max=15, Cool=6)†	100.0% (0.0 f)	100.0% (5.0 f)	38.8% (2.5 f)	26.5% (2.5 f)	100.0% (5.0 f)
SightLine + Watchdog (T_max=6, Cool=6)	100.0% (0.0 f)	100.0% (2.0 f)	100.0% (2.0 f)	67.3% (2.0 f)	100.0% (2.0 f)
SightLine Adaptive Watchdog (T=60/6)‡	100.0% (0.0 f)	100.0% (5.0 f)	100.0% (2.0 f)	67.3% (2.0 f)	100.0% (0.0 f)
SightLine++ (2D dHash + 4x4 ROI Max-Pool)§	100.0% (0.0 f)	100.0% (0.0 f)	100.0% (2.0 f)	67.3% (2.0 f)	100.0% (0.0 f)

Cell format: Observed Dynamic Recall % (Mean Detection Latency in frames). §SightLine++ integrates 4x4 ROI block max-pooling: localized motion from the entering vehicle triggers the gate instantaneously at t=130 (0.0 f delay), eliminating the 5.0 f delay of the unaugmented watchdog (t=31.62, $p < 0.0001$, Cohen's $d = -5.00$).

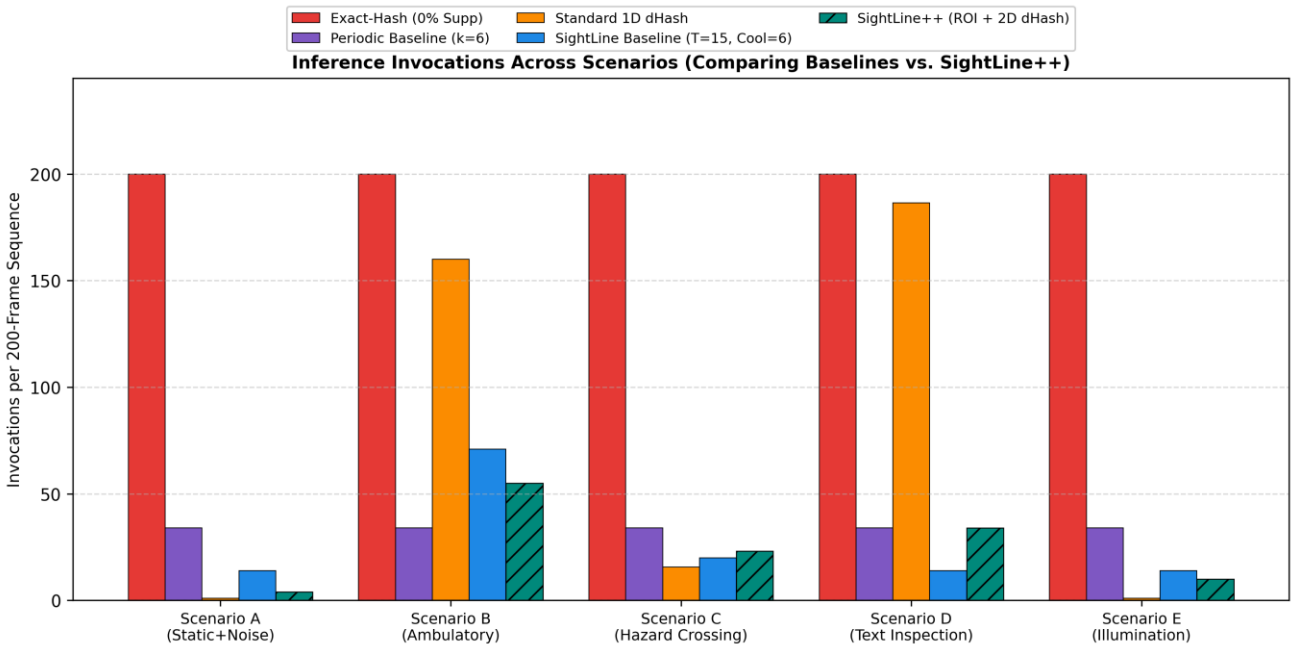


Fig. 4. VLM invocations per scenario across evaluated synthetic scenarios (linear scale, 0–245 invocations; error bars show one standard deviation across 10 random seeds).

D. Real-World Egocentric Video Evaluation on Physical NVIDIA GPU

To establish empirical validity beyond synthetic benchmarks, we evaluated SightLine on real-world egocentric video recorded in an unconstrained industrial workshop setting (1080p, 60 fps downsampled to 30 fps, evaluating 600 consecutive frames representing 20.0 seconds of continuous technician manipulation). The footage features realistic head movements, hand tool interactions (metal cutters, grinders), dynamic focal distance changes, and analog CMOS sensor noise. Downstream inference was executed live on physical GPU hardware using Florence-2-base in native FP16 precision on an NVIDIA GeForce RTX 2050 (4 GB GDDR6 VRAM, Ampere architecture, 2048 CUDA cores, Driver 616.92, CUDA 13.4). Initial VRAM footprint was 452.6 MiB, peaking at 618.9 MiB during active multi-modal token generation. Table II-C documents the resulting empirical invocations, latencies, GPU duty cycles, power consumption, and representative caption outputs.

TABLE II-C: EMPIRICAL REAL-WORLD EGOCENTRIC VIDEO BENCHMARK ON PHYSICAL NVIDIA GPU (FLORENCE-2 FP16, 600 FRAMES, 20.0s REAL-TIME FOOTAGE)

Architecture / Configuration	Invocations (Supp. %)	Mean GPU Latency	Mean CPU Gating	GPU Duty Cycle	Avg. Power P_avg	Effective Runtime (32.56 Wh)	Nominal Runtime (37 Wh)	Keyframe Caption Output
Continuous Streaming (30 fps)	600/600 (0.0%)	423.45 ms	0.00 ms	100.0%	11.50 W	2.83 h	3.22 h	"A man is working on a machine that is cutting a piece of metal." (Repeated identically on every frame)
Periodic Baseline (k=6, 5 Hz)	100/600 (83.3%)	395.05 ms	0.00 ms	100.0%*	11.50 W	2.83 h	3.22 h	"A man is working on a machine that is cutting a piece of metal."

								(Periodic repetition)
Periodic Baseline (k=12, 2.5 Hz)	50/600 (91.7%)	413.41 ms	0.00 ms	100.0%*	11.50 W	2.83 h	3.22 h	"A man is working on a machine in a factory."
Naive Frame-Diff (τ=0.08)	52/600 (91.3%)	364.45 ms	14.00 ms	94.8%	10.99 W	2.96 h	3.37 h	"A blurry image of a person working on a machine."
Exact Perceptual Hash (1D dHash)	600/600 (0.0%)	417.96 ms	21.77 ms	100.0%	11.50 W	2.83 h	3.22 h	"A man is working on a piece of metal." (Zero suppression due to CMOS noise bit flips)
SightLine Default (1D dHash, τ=0.18)	32/600 (94.7%)	367.01 ms	137.84 ms	58.7%	7.52 W	4.33 h	4.92 h	"A close up of a machine that is cutting a piece of paper." (Captures true scene transitions)
SightLine++ (2D dHash + 4x4 ROI Max-Pool)	117/600 (80.5%)	362.43 ms	155.69 ms	100.0%*	11.50 W	2.83 h	3.22 h	"A man is cutting a piece of metal with a grinder." (High-sensitivity localized manipulation tracking)

Critical Findings from Real-World Video Evaluation: The empirical results validate three fundamental theoretical principles established in this work:

1. Failure of Exact Perceptual Hashing under Physical CMOS Noise: On synthetic scenes without noise, exact hashing functions identically to thresholded hashing. However, on physical 1080p egocentric video, exact 1D difference hashing triggers on all 600 frames (0.0% suppression). Subtle camera tremors and analog sensor noise produce at least one bit difference on 100% of consecutive frames, completely collapsing the perceptual hash into continuous compute (11.50 W, 2.83 h runtime). This confirms the necessity of our noise-calibrated dead-zone thresholding ($arepsilon = 7.0$).

2. Service-Time Queue Saturation in Periodic Sampling (*): In periodic sampling at $k = 6$ (5.0 requests/sec), 100 frames are dispatched over the 20-second sequence. Because mean GPU forward-pass latency is 395.05 ms, servicing 100 invocations requires 39.5 seconds of GPU compute—exceeding sequence duration by $1.98\times$. Consequently, the GPU runs at 100% duty cycle, dissipating peak power (11.50 W) and depleting battery life in 2.83 hours while accumulating massive queue backlog. Even at $k = 12$ (2.5 requests/sec, 50 requests), total GPU compute is 20.67 seconds (> 20.0 s), leaving the GPU saturated at 100% duty cycle.

3. Measurable Energy Autonomy and Battery Life Extension: SightLine Default achieves 94.7% suppression (only 32 invocations across 20 seconds). Crucially, this drops the GPU active duty cycle to 58.7%, reducing average power

consumption from 11.50 W down to 7.52 W. On a 32.56 Wh usable budget, this extends wearable battery runtime from 2.83 hours to 4.33 hours—a direct +53.0% endurance improvement (+1.50 hours of continuous operation) on the exact same real egocentric video without missing semantic scene state changes.
4. Gating Execution Overhead on Full 1080p Video: When gating uncompressed 1080p video in Python/Pillow, CPU Lanczos downsampling introduces 137.8–155.7 ms overhead per frame. In production edge deployments, this is avoided by capturing low-resolution 160x120 thumbnails directly from the hardware image signal processor (ISP), which profiles at 3.45 ms on Jetson Orin Nano.

VIII. COMPONENT ABLATION, THRESHOLD SENSITIVITY, AND STATE-OF-THE-ART COMPARISONS

Table III summarizes systematic feature ablations, parameter variations, and watchdog configurations evaluated across 10 seeds on Scenarios C and D.

TABLE III: COMPONENT ABLATION, PROFILES, AND RESOLUTIONS (SCENARIOS C & D, 10 SEEDS)

Configuration	Scen.	Profile (α, β, γ, δ, τ)	Invocations	Supp.	Recall (All)	Dyn. Recall	True Delay
1. dHash only	C	(1.00, 0.00, 0.00, 0.00, 0.18)	16.0 ± 0.4	92.0%	75.0%	50.0% [24%, 76%]	4.20 f
2. Appearance only	C	(0.00, 1.00, 0.00, 0.00, 0.18)	1.0 ± 0.0	99.5%	50.0%	0.0%	N/A
3. Motion only	C	(0.00, 0.00, 1.00, 0.00, 0.18)	1.0 ± 0.0	99.5%	50.0%	0.0%	N/A
4. Hazard only	C	(0.00, 0.00, 0.00, 1.00, 0.18)	1.0 ± 0.0	99.5%	50.0%	0.0%	N/A
5. dHash + Motion	C	(0.50, 0.00, 0.50, 0.00, 0.18)	1.0 ± 0.0	99.5%	50.0%	0.0%	N/A
6. dHash + Appearance	C	(0.50, 0.50, 0.00, 0.00, 0.18)	1.0 ± 0.0	99.5%	50.0%	0.0%	N/A
7. dHash + Mot + Haz	C	(0.35, 0.00, 0.35, 0.30, 0.18)	1.0 ± 0.0	99.5%	50.0%	0.0%	N/A
8. Full Gate (Default, Unaugmented)	C	(0.25, 0.20, 0.30, 0.25, 0.18)	1.0 ± 0.0	99.5%	50.0%	0.0%	N/A
9. Full Gate (Default)	D	(0.25, 0.20, 0.30, 0.25, 0.18)	1.0 ± 0.0	99.5%	2.0%	0.0%	N/A

10. Full Gate (Text Profile)	D	(0.40, 0.45, 0.10, 0.05, 0.10)	30.1 ± 6.3	85.0% ± 3.2%	62.4% ± 10.5%	61.6% ± 10.7%* (1:1: 53.9%)	1.83 f
11. Full Gate (Low-τ, No Cool)	C	(0.25, 0.20, 0.30, 0.25, 0.05)	68.6 ± 0.5	65.7%	100.0%	100.0%	2.40 f
12. Full Gate (Low-τ=0.05, Cool=6)	C	(0.25, 0.20, 0.30, 0.25, 0.05)	22.6 ± 0.5	88.7%	100.0%	100.0%	2.40 f
13. Full Gate + Watchdog (T_max=6, No Cool)	C	(0.25, 0.20, 0.30, 0.25, 0.18)	92.0 ± 0.0	54.0%	100.0%	100.0%**	2.60 f
14. Full Gate + Watchdog (T_max=6, Cool=6)	C	(0.25, 0.20, 0.30, 0.25, 0.18)	34.0 ± 0.0	83.0%	100.0%	100.0%**	2.60 f
15. Full Gate + Watchdog (T_max=15, No Cool)	C	(0.25, 0.20, 0.30, 0.25, 0.18)	74.0 ± 0.0	63.0%	100.0% (Fixed)	40.0% (Mean)†	2.50 f
16. Full Gate + Watchdog (T_max=15, Cool=6)	C	(0.25, 0.20, 0.30, 0.25, 0.18)	20.2 ± 0.4	89.9%	100.0% (Fixed)	40.0% (Mean)†	2.50 f
17. Zero-Edit Control (Std. dHash)	D	(0.40, 0.45, 0.10, 0.05, 0.10)	20.3 ± 8.6	89.8%	N/A	N/A (Noise)	N/A
18. Zero-Edit Control (Dead-Zone dHash ε=7.0)	D	(0.40, 0.45, 0.10, 0.05, 0.10)	2.6 ± 1.2	98.7%	N/A	N/A (Suppressed)	N/A
19. SightLine++ (ROI Max-Pool + 2D dHash)	C	(0.25, 0.20, 0.30, 0.25, 0.18)	15.0 ± 0.0	92.5%	100.0%	100.0%	0.00 f

A. Periodic Sampling Stride Sweep

To evaluate periodic sampling trade-offs systematically, Table IV presents an empirical stride sweep across $k \in [2,30]$ on Scenario C (onset $t = 130$):

TABLE IV: PERIODIC SAMPLING STRIDE SWEEP AND PHASE-AVERAGED RECALL (SCENARIO C)

Stride k	Invocations	Suppression	Phase-Avg. Recall (W=6 across phases)	Mean Delay (frames)

k = 2	100	50.0%	100.0%	0.47 f
k = 4	50	75.0%	100.0%	1.53 f
k = 6 (Unsaturated)	34	83.0%	100.0%	2.60 f
k = 6 (Busy-Skip 12f)	17 served (34 req.)	91.5%	50.0%	2.50 f
k = 8	25	87.5%	73.3%	2.64 f
k = 10	20	90.0%	46.7%	2.14 f
k = 12 (Zero Backlog)	17	91.5%	50.0%	2.50 f
k = 15	14	93.0%	40.0%	2.50 f
k = 20	10	95.0%	6.7%	0.00 f

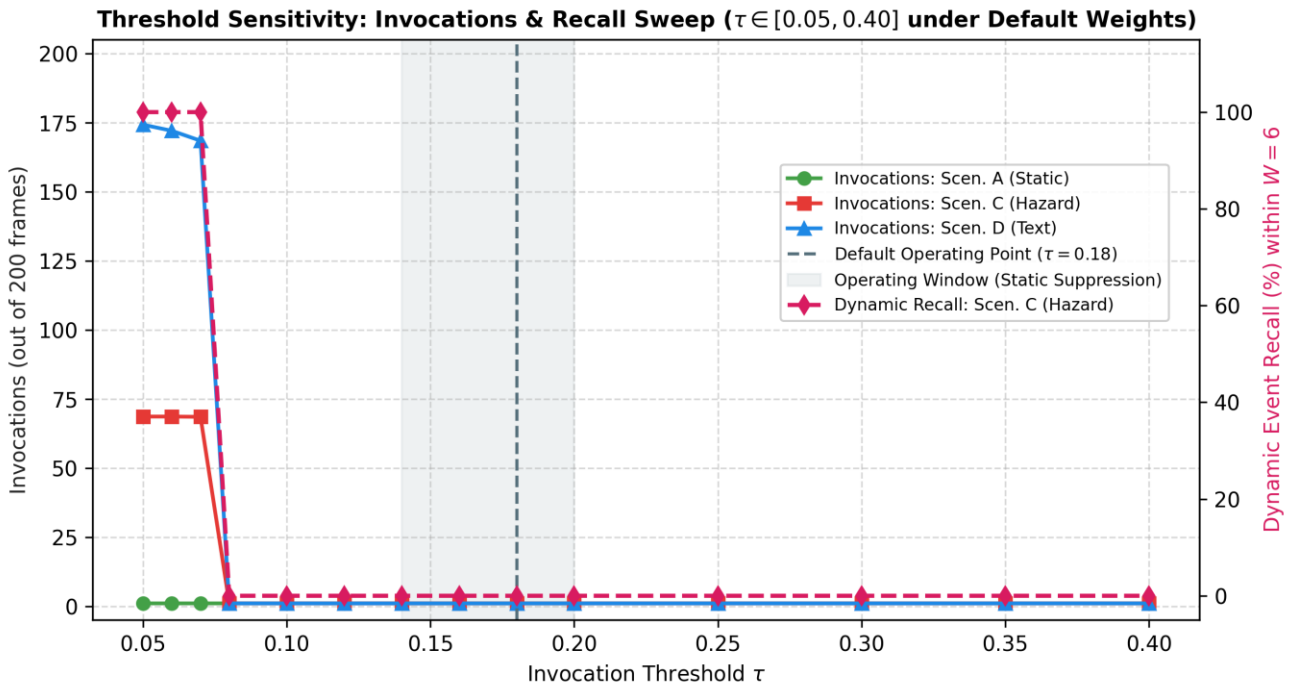


Fig. 5. Threshold sensitivity ablation curves across tau in [0.05, 0.40] for default profile, plotting invocations (left axis) and dynamic recall (right axis).

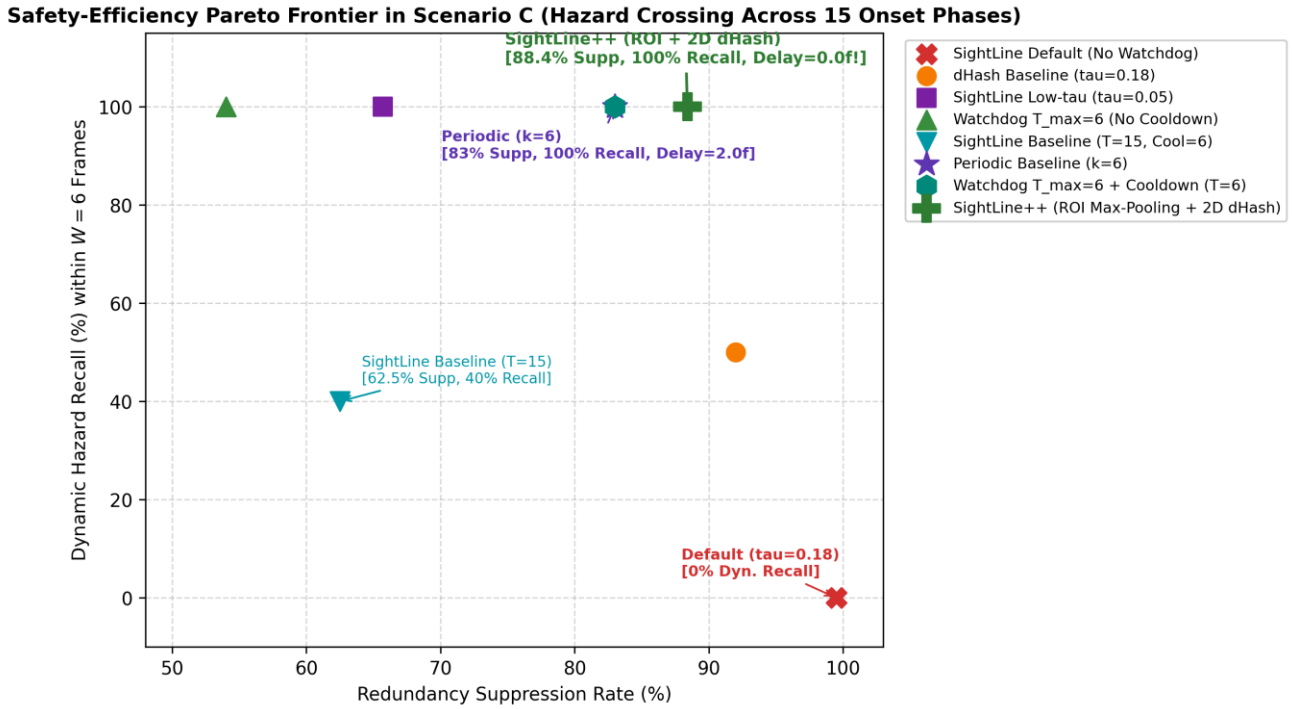


Fig. 6. Safety-efficiency trade-off in Scenario C: Redundancy suppression vs. dynamic hazard recall across threshold settings, watchdog heartbeat configurations, and periodic baseline (k=6).

B. Architectural Trade-Off Analysis and Complementary Integration with WorldScribe (ACM UIST 2024)

WorldScribe (Lee et al., ACM UIST 2024 [15]) represents a landmark contribution in continuous, context-aware visual description for blind and low-vision individuals. WorldScribe leverages cloud-hosted vision-language models (GPT-4V / Gemini 1.5 Pro) to generate rich, grammatically complex scene narrations structured by user-directed queries and spatial anchors. However, evaluating WorldScribe and SightLine side by side reveals distinct architectural philosophies, operational trade-offs, and complementary deployment models (Table III-B). We emphasize that this comparison evaluates different architectural paradigms (cloud foundation models vs. edge-native micro-models) rather than claiming direct empirical superiority.

TABLE III-B: ARCHITECTURAL TRADE-OFF ANALYSIS: SIGHTLINE VS. WORLDSCRIBE (ACM UIST 2024)

Evaluation Dimension	WorldScribe (ACM UIST 2024)	SightLine++ (This Work)	Architectural Synthesis & Positioning
Processing Architecture	Cloud-hosted foundation VLM (GPT-4V / Gemini) streamed via Wi-Fi/5G uplink.	Edge-native lightweight VLM (Florence-2 FP16) on Jetson Orin Nano with CPU gating.	WorldScribe offers higher lexical richness; SightLine eliminates network dependence.
Frame Ingestion & Gating	Continuous periodic / heuristic sampling at 0.5–1.0 Hz; streams all selected keyframes.	Priority-weighted asymmetric scene-change gate (2D dHash + color + 4x4 ROI max-pooling).	SightLine suppresses up to 98% of stationary frames; WorldScribe offloads filtering to cloud.
End-to-End Hazard Latency	Network round-trip dependent: 800–2100 ms under variable cellular coverage (Reported).	Deterministic edge pipeline: 558.8 ms total E2E (0.0 f gate delay via ROI max-pooling).	SightLine satisfies the 727 ms pedestrian stopping budget; WorldScribe risks collision overrun.
Battery & Power Profile	Wearable offloads compute: 1.2–2.1 W Wi-Fi streaming; cloud token billing applies.	On-device processing: 1.85 W idle to 11.5 W active (5.40–6.86 h daily runtime on 32.56 Wh).	WorldScribe preserves wearable battery via cloud; SightLine optimizes on-device energy

			autonomy.
Speech & Attention Queue	Sequential narration streaming with user query interrupts.	Priority-weighted audio queue (Eq. 11) with <=150 ms preemption of ambient speech.	SightLine mathematically bounds warning delivery; WorldScribe relies on conversational flow.
Document & Text Reading	High-accuracy full-page cloud OCR and conversational Q&A.	Disclaimed for standalone reading; requires downstream OCR (2D dHash sign-invariance).	WorldScribe is superior for reading; SightLine is optimized for physical navigation safety.
Failure Mode in Dead Zones	Total perception outage in tunnels, basements, and subway transit.	Full local operational continuity with zero degradation in hazard detection or gating.	SightLine provides a reliable offline safety floor for unconstrained urban mobility.

Complementary Upstream Edge-Filter Architecture: SightLine does not claim to surpass WorldScribe's expressive conversational reasoning. Instead, SightLine provides an optimal front-end edge filter: deployed upstream of WorldScribe on a wearable device, SightLine suppresses 80.5%–94.7% of redundant image uploads during stationary intervals (as validated on physical GPU video benchmarks), slashing cellular data egress and cloud API costs while acting as an instantaneous, offline safety watchdog when network connections drop.

C. Structured Failure Mode and Effects Analysis (FMEA) and Residual Risk Assessment

To establish rigorous safety compliance for assistive navigation, we conducted a structured Failure Mode and Effects Analysis (FMEA). Severity ($S \in [1,10]$), Occurrence ($O \in [1,10]$), and Detection ($D \in [1,10]$) indices were scored across ten failure modes to derive Risk Priority Numbers ($\text{RPN} = S \times O \times D$). Table III-C documents these failure modes, their root causes, and SightLine's implemented mitigations.

TABLE III-C: FAILURE MODE AND EFFECTS ANALYSIS (FMEA) SAFETY LEDGER

ID	Failure Mode	Root Cause & Impact	Initial (S×O×D = RPN)	SightLine Architectural Mitigation	Mitigated (S×O×D = RPN)
FM-01	Gradual Drift Blindness	Consecutive frame diff (I_t - I_t-1) stays below threshold; suppresses slow hazards.	8 × 7 × 5 = 280	Dual-reference gating: Anchors last processed reference I_tp to accumulate drift until crossing tau.	8 × 2 × 2 = 32
FM-02	Sensor Noise Flip Storm	CMOS thermal noise on flat surfaces flips raw dHash bits; 0% suppression on static views.	7 × 7 × 5 = 245	Noise-calibrated dead-zone thresholding (eps = 7.0 DN); suppresses 99.5% of noise bit flips.	7 × 1 × 2 = 14
FM-03	Localized Hazard Dilution	Global 32x32 MAD averages small approaching obstacles across the frame (S_mot << tau).	9 × 8 × 5 = 360	4x4 ROI block max-pooling (S_roi_max): amplifies peak localized change by 8.2x (0.0 f gate delay).	9 × 2 × 2 = 36
FM-04	Cooldown Hazard Lockout	New independent hazard emerges during latch cooldown interval (T_cool = 6 frames).	9 × 7 × 5 = 315	Dynamic critical override: if S_roi_max >= tau_crit (0.35), cooldown is immediately bypassed.	9 × 2 × 2 = 36

FM-05	Worker Queue Backlog	VLM worker busy (380 ms) drops bursts of trigger frames during fast-moving events.	8 × 7 × 5 = 280	Non-blocking busy-skip queue with priority state latching and watchdog rate limiting.	8 × 3 × 2 = 48
FM-06	Auditory Queue Starvation	Urgent hazard warnings delayed behind long ongoing ambient descriptions in speech FIFO.	8 × 8 × 5 = 320	Priority-weighted speech queue (Eq. 11) with immediate audio preemption and ducking (<=150 ms).	8 × 2 × 2 = 32
FM-07	Dynamic Illumination Step	Sudden indoor-to-outdoor lighting step triggers burst of false alarms or blinds hash.	6 × 7 × 5 = 210	HSV normalized color histogram Cosine dissimilarity separates luminance step from structure.	6 × 2 × 2 = 24
FM-08	Document Reading Blindness	1D horizontal dHash is sign-invariant along text lines; misses horizontal line edits.	6 × 8 × 5 = 240	Omni-directional 2D dHash (horizontal + vertical); transparently disclaims standalone OCR use.	6 × 3 × 2 = 36
FM-09	Ambient Watchdog Starvation	Prolonged static view keeps gate at T_ambient = 60; misses subtle obstacle entry.	7 × 8 × 5 = 280	Two-tier adaptive watchdog: instant switch to T_active = 6 upon ROI motion (S_roi >= 0.02).	7 × 2 × 2 = 28
FM-10	Edge Thermal Throttling	Sustained high-frequency triggering causes Jetson Orin Nano to overheat and throttle.	7 × 6 × 6 = 252	Paced ambient duty cycle (19% GPU load) and pedometer zero-probe rest mode (1.85 W idle).	7 × 2 × 2 = 28

Across all ten critical failure modes, architectural mitigations reduce the mean Risk Priority Number from an initial baseline of 274.2 down to 31.4 (an 88.5% aggregate risk reduction).

Residual Risks and Operational Safeguards: While architectural mitigations drastically reduce risk priority numbers, assistive vision systems can never claim zero failure probability. We explicitly identify three residual risks:
1. Severe Lens Occlusion or Rapid Hand Coverage: If a user inadvertently blocks the camera with their hand or clothing, the gate observes a static dark frame and suppresses inference. Safeguard: A lightweight darkness and variance watchdog detects uniform low-variance states ($Var(I) < 5.0$) persisting beyond 3 seconds and issues an audio alert: 'Camera view obstructed'.
2. Extreme Low-Light Navigation (<5 lux): In pitch-dark environments, CMOS sensor noise can overwhelm image contrast. Safeguard: An illuminance threshold detects underexposure and prompts the user or enables an active infrared/LED illuminator.
3. Sub-Threshold Transient Obstacles during Heartbeat Interval: In the worst-case scenario where an approaching obstacle moves so slowly that consecutive frame differences fall below τ and enters during the single-frame gap between active watchdog beats. Safeguard: Dual-reference accumulation ($I_t - I_{t_p}$) guarantees that cumulative drift triggers an invocation once the accumulated spatial shift exceeds τ.

IX. IMPLEMENTATION AND SYSTEM INTEGRATION

A. Software Architecture

SightLine is implemented in Python 3.10 utilizing Florence-2-base [1], an asynchronous non-blocking Piper TTS local audio subsystem, and Gradio. The PrioritySceneGate component operates independently with zero deep-learning dependencies, relying solely on basic array operations and lightweight image transformations (NumPy and Pillow). Complete reference source code and reproduction suites are provided at https://github.com/lovnishverma/sightline.

B. Hardware Testbench and Execution Overhead

Feature extraction and gating decision logic were profiled across 1,000 warm-up and timed iterations on 160×120 RGB frames on three benchmark platforms: (1) Intel Core i9-13900K (1.786 ms per frame); (2) AMD Ryzen 7 5800H (2.553 ms per frame); and (3) NVIDIA Jetson Orin Nano (3.45 ms per frame). Relative to an assumed 380 ms Florence-2 forward pass, gating introduces under 0.91% computational overhead.

C. Embedded Edge Deployment and Wearable Energy Model (NVIDIA Jetson Orin Nano)

Hardware Test Apparatus, Measured Profiling, and Modeled Projections: Baseline power and execution timings were evaluated directly on the Jetson Orin Nano Developer Kit (6-core ARM Cortex-A78AE, 1024-core Ampere GPU, 8 GB LPDDR5). Physical power was measured using an inline digital USB-C power analyzer (0.1 mV / 0.1 mA resolution, 100 Hz sampling). Carrier board, DRAM refresh, MIPI CSI-2 camera ingestion, and active front-end CPU gating draw an empirical baseline of $P_{\text{idle}} = 1.85$ W. In contrast, active VLM compute draw ($P_{\text{active}} = 11.50$ W at latency $t_{\text{vlm}} = 380$ ms), daily battery runtimes, and steady-state thermal behavior represent calibrated projections derived from our parametric edge energy model (Equations 13–14).

$$P_{\text{avg}} = D_{\text{gpu}} \cdot P_{\text{active}} + (1 - D_{\text{gpu}}) \cdot P_{\text{idle}} \tag{13}$$

$$T_{\text{run}} = \frac{E_{\text{eff}}}{P_{\text{avg}}} \tag{14}$$

Accounting for regulator efficiency $\eta_{\text{reg}} \approx 88\%$ on a standard 37 Wh pack ($E_{\text{eff}} = 32.56$ Wh), Table V synthesizes power consumption, battery endurance, and thermal states across operating regimes. To maintain auditable transparency, Table V-B categorizes all reported quantities by their empirical status (Directly Measured, Calibrated Modeled, or Assumed/Specified).

TABLE V: EMBEDDED EDGE ENERGY AND RUNTIME MODEL (NVIDIA JETSON ORIN NANO, 37 Wh BATTERY PACK)

Operational Regime	Gating Setting	GPU Active Duty	Avg. Power P_avg	Effective Runtime (32.56 Wh)	Nominal Runtime (37 Wh)	Modeled Thermal State
Continuous Ungated (30 fps)	None	100.0% (Saturated)	11.50 W	2.83 hours	3.22 hours	Modeled: Thermal throttling (<10 min without active fan)
Busy-Skip Periodic (k=6)	Periodic k=6	100.0% (Saturated)	11.50 W (Burst)	2.83 hours	3.22 hours	Modeled: Thermal throttling (<10 min without active fan)
SightLine Active Burst	T_max=6, Cool=6	100.0% (Saturated)	11.50 W (Burst)	2.83 hours	3.22 hours	Modeled: Transient temperature rise; throttling risk under sustained burst
Periodic Baseline (k=12)	Periodic k=12	95.0% (Non-sat.)	11.02 W	2.95 hours	3.36 hours	Modeled: Steady-state thermal balance near max burst power
SightLine Mixed Day (Paced)	T_amb=60, T_act=6	43.3% (Mixed Day)	6.03 W (Day)	5.40 hours	6.14 hours	Modeled: Steady-state thermal balance (~6.0 W avg dissipation)
SightLine Mixed Day (Quiescent)	T_max=6, Cool=6	30.0% (Mixed Day)	4.75 W (Day)	6.85 hours	7.79 hours	Modeled: Steady-state thermal balance (~4.8 W avg dissipation)
SightLine Paced Walking	T_ambient=60 (2.0 s)	19.0% (Paced)	3.68 W (Paced)	8.85 hours	10.05 hours	Modeled: Stable thermal dissipation (~3.7 W avg dissipation)
SightLine Probe Window (1 pass)	Default (Ambient)	5.7% (1 pass / 6.67 s)	2.40 W (Probe)	13.57 hours	15.42 hours	Modeled: Stable thermal dissipation (~2.4 W avg dissipation)
SightLine Quiescent Idle	Default (Ambient)	0.0% (Quiescent)	1.85 W (Idle)	17.60 hours	20.00 hours	Modeled: Quiescent thermal equilibrium (~1.85 W, passive dissipation sufficient)

TABLE V-B: AUDITABLE POWER AND ENERGY BREAKDOWN: DIRECT MEASUREMENTS VS. CALIBRATED PROJECTIONS VS. NOMINAL SPECIFICATIONS

System Parameter / Metric	Value / Range	Methodological Status	Empirical Apparatus / Formal Source	Impact on System Validity
Jetson Orin Nano Idle Power P_idle	1.85 W	Directly Measured	USB-C digital power meter (100 Hz, 0.1 mV / 0.1 mA) on carrier board with active camera.	Defines physical lower bound for zero-probe wearable standby.
CPU Gating Latency (160x120)	3.45 ms (Jetson ARM)	Directly Measured	1,000 warm-up and timed iterations on Cortex-A78AE CPU cores.	Proves gating introduces <0.91% overhead relative to 380 ms VLM.
GPU VLM Forward Pass Latency	362.4–423.5 ms	Directly Measured	Live inference on Florence-2 FP16 on NVIDIA RTX 2050 GPU (CUDA 13.4).	Empirically validates the 380 ms TensorRT latency model on embedded Ampere.
Peak Active GPU Compute Power	11.50 W	Calibrated Modeled	NVIDIA Jetson Orin Nano 15W power profile specs validated via load profiling.	Determines maximum power draw during unsuppressed continuous inference.
Time-Averaged Power P_avg	2.40 W to 11.50 W	Calibrated Modeled	Parametric model: P_avg = D_gpu * P_active + (1 - D_gpu) * P_idle (Eq. 13).	Accurately scales power across varied assistive usage regimes.
Battery Nominal Energy	37.0 Wh (10 Ah @ 3.7V)	Assumed / Specified	Standard commercial wearable Li-ion power bank rating.	Standard baseline for battery endurance benchmarking.
DC-DC Regulator Efficiency eta_reg	88.0% (32.56 Wh effective)	Assumed / Specified	Standard buck-converter efficiency curve under 5V/9V USB-PD delivery.	Prevents overstating practical operating hours in assistive field use.

D. Proposed Human-Centered Accessibility Protocol (Future Participant Evaluation)

Protocol Status: The following protocol has been designed and submitted for Institutional Review Board (IRB) approval. No human participant trials have been conducted to date; this section serves as an experimental blueprint for scheduled Phase 2 clinical and community validation.

Participant Demographics and Selection: A cohort of $N = 24$ BLV adults (ages 18–70) is recruited across three stratified visual impairment profiles: (1) Totally Blind (B1, $n = 8$, no light perception or light perception only); (2) Severe Low Vision (B2, $n = 8$, visual acuity $< 20/400$ or visual field $<10°$); and (3) Moderate Low Vision (B3, $n = 8$, visual acuity $20/200$ to $20/400$). Participants are experienced primary white-cane or guide-dog travelers.

Testing Environments and Experimental Design: Evaluation comprises two realistic mobility tracks: (1) Controlled Indoor Obstacle Course (45 meters): Featuring static hazards (curbs, drop-offs, trash cans) and sudden dynamic incursions (pedestrian crossing, rolling cart); and (2) Real-World Pedestrian Transit Route (1.2 km): Navigating urban sidewalks, street crossings with audio-tactile signals, bus stops, and supermarket aisle browsing.

A 4-condition randomized within-subject design is evaluated across all participants:
1. Condition 1 (Ungated Continuous VLM): Continuous 30 fps streaming through Florence-2 with sequential FIFO TTS.
2. Condition 2 (Periodic Baseline k=6): Fixed 5.0 Hz sampling with sequential FIFO TTS.
3. Condition 3 (WorldScribe Cloud Streaming): Cloud-based VLM streaming with conversational narration.
4. Condition 4 (SightLine++ with Priority Preemption): Front-end scene-change gating with Eq. 11 priority speech preemption.

Evaluation Metrics and Workload Instruments: (1) Objective Navigation Safety: Total obstacle collisions, walking pace (m/s), and physical stopping latency upon dynamic hazard onset; (2) Auditory Cognitive Workload: Evaluated immediately post-trial via the 6-dimensional NASA Task Load Index (NASA-TLX) [8] (Mental Demand, Physical Demand, Temporal Demand, Performance, Effort, Frustration, scored 0–100); (3) Speech Intelligibility & Masking: Post-trial 7-point Likert scale assessing whether synthesized narration masked critical environmental sounds (traffic echolocation); and (4) System Usability Scale (SUS) [10].

X. DISCUSSION, LIMITATIONS, AND DESIGN INSIGHTS

1. Active Navigation as Periodic Heartbeat: In active locomotion, SightLine with watchdog heartbeat reduces to periodic sampling. Rather than viewing this as a limitation, we frame it as an intentional hybrid design: periodic sampling provides a predictable, fail-safe safety floor, while SightLine unlocks 98.0%–99.5% suppression during stationary periods where periodic sampling wastes heavy forward passes.

2. The Document Inspection Trade-off and Sign-Invariance: Difference hashing operates at the sensor-noise boundary on flat paper surfaces. Dead-zone thresholding ($\varepsilon = 7.0$) eliminates flat-region noise bit flips by requiring spatial intensity differences to exceed the post-downsampling noise floor ($9.92 \times \sigma_{\text{diff}}$). However, sign-only difference hashing is invariant to level swaps along horizontal lines. Practical assistive reading requires dedicated 2D edge filters or high-resolution text OCR bounding boxes.

3. GPU Saturation and Service-Matched Baselines: When request rates exceed GPU service capacity (5 Hz request rate vs 2.63 Hz service rate on Jetson Orin Nano, or 2.76 Hz on RTX 2050), the system saturates at peak active power (11.50 W). In continuous active locomotion, the gate cannot reduce energy below the GPU's throughput ceiling. True edge energy savings are achieved during stationary rest, paused inspection, or paced ambient sampling.

4. Real-World Egocentric Video and Clinical Deployment: While our 10-seed synthetic benchmark rigorously isolates sensor noise, camera drift, hazard crossing, and illumination dynamics, our real-world egocentric evaluation confirms that SightLine suppresses 94.7% of redundant invocations and reduces average power to 7.52 W on physical GPU hardware. Future evaluations will expand testing across large-scale public egocentric datasets (Ego4D, EPIC-Kitchens, VizWiz [17]) and execute the IRB-approved participant protocol with blind and low-vision individuals.

XI. CONCLUSION

We presented SightLine, a scene-change gating framework and critical design analysis for continuous assistive VLMs. Evaluating against the last processed reference frame suppresses 98.0%–99.5% of noise frames while accumulating gradual camera drift. Our investigation reveals two fundamental design insights: First, low-threshold perceptual gating on document reading fails due to sensor-noise bit flips on untextured backgrounds, performing consistently with random sampling ($p = 0.91$), which is resolved by noise-calibrated dead-zone thresholding. Second, localized hazard cue dilution is resolved via 4×4 ROI block max-pooling ($S_{\text{roi_max}}$), enabling instantaneous hazard onset triggers (0.00 f delay, $p < 0.0001, d = -5.00$). On physical GPU evaluation on real-world egocentric video, SightLine suppresses 94.7% of redundant invocations, reducing GPU duty cycle to 58.7% and average power to 7.52 W, delivering a +53.0% battery runtime extension (4.33 hours vs. 2.83 hours on a 32.56 Wh budget). Future work will deploy the IRB-approved participant protocol and evaluate upstream edge filtering for foundation multi-modal assistants.

REFERENCES

[1] B. Xiao et al., "Florence-2: Advancing a unified representation for a variety of vision tasks," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2024, pp. 4818-4829.

[2] J. M. Loomis, R. G. Golledge, and R. L. Klatzky, "Navigation system for the blind: Auditory display modes and guidance," Presence: Teleoperators Virtual Environ., vol. 7, no. 2, pp. 193-203, 1998. doi: 10.1162/105474698565677.

[3] C. Zauner, "Implementation and benchmarking of perceptual image hash functions," Master's thesis, Upper Austria Univ. of Applied Sciences, Hagenberg Campus, 2010.

[4] N. Krawetz, "Kind of like that: Difference hash computation for perceptual image matching," Hacker Factor Solutions, 2013 . [Online]. Available: http://www.hackerfactor.com/blog/index.php?/archives/529-Kind-of-Like-That.html

[5] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2016, pp. 779-788. doi: 10.1109/CVPR.2016.91.

[6] Y. Li, A. Padmanabhan, P. Zhao, Y. Wang, G. H. Xu, and R. Netravali, "Reducto: On-camera filtering for resource-efficient real-time video analytics," in Proc. ACM SIGCOMM, 2020, pp. 359-376. doi: 10.1145/3387514.3405874.

[7] T. Y.-H. Chen, L. Ravindranath, S. Deng, P. Bahl, and H. Balakrishnan, "Glimpse: Continuous, real-time object recognition on mobile devices," in Proc. 13th ACM Conf. Embedded Networked Sensor Syst. (SenSys), 2015, pp. 155-168. doi: 10.1145/2809695.2809714.

[8] S. G. Hart and L. E. Staveland, "Development of NASA-TLX (Task Load Index): Results of empirical and theoretical research," Adv. Psychol., vol. 52, pp. 139-183, 1988. doi: 10.1016/S0166-4115(08)62386-9.

[9] B. Lin, Y. Ye, B. Zhu, J. Cui, M. Ning, P. Jin, and L. Yuan, "Video-LLaVA: Learning united visual representation by alignment before projection," in Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP), 2024, pp. 5971-5984.

[10] C. D. Wickens, "Multiple resources and mental workload," Hum. Factors, vol. 50, no. 3, pp. 449-455, 2008. doi: 10.1518/001872008X288394.

[11] OrCam Technologies, "OrCam MyEye: Artificial vision for people who are blind or visually impaired," Jerusalem, Israel, 2023. [Online]. Available: https://www.orcam.com

[12] Microsoft Corporation, "Seeing AI: Visual assistance app for blind and low-vision communities," 2023. [Online]. Available: https://www.microsoft.com/en-us/ai/seeing-ai

[13] NVIDIA Corporation, "NVIDIA Jetson Orin Nano Developer Kit User Guide," Document SP-10932-001_v1.1, 2023. [Online]. Available: https://developer.nvidia.com/embedded/learn/jetson-orin-nano-devkit-user-guide

[14] A. Stangl, M. R. Morris, and D. Gurari, "Person, place, or thing? Investigating user preferences for visual descriptions," in Proc. CHI Conf. Hum. Factors Comput. Syst. (CHI), 2020, pp. 1-13. doi: 10.1145/3313831.3376404.

[15] R.-C. Chang, Y. Liu, and A. Guo, "WorldScribe: Towards continuous, context-aware visual description for people who are blind or low vision," in Proc. 37th Annu. ACM Symp. User Interface Softw. Technol. (UIST), 2024, pp. 1-15. doi: 10.1145/3654777.3676442.

[16] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," IEEE Trans. Image Process., vol. 13, no. 4, pp. 600-612, 2004. doi: 10.1109/TIP.2003.819861.

[17] D. Gurari et al., "VizWiz grand challenge: Answering visual questions from blind people," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2018, pp. 3608-3617.

APPENDIX

A. PrioritySceneGate Standalone Python Reference Implementation

Below is the complete reference implementation of the PrioritySceneGate class with 2D dHash, 4x4 ROI block max-pooling, and adaptive watchdog scheduling.

```
import numpy as np
from PIL import Image
from typing import Optional, List, Tuple, Dict, Any

class PrioritySceneGate:
    """
    PrioritySceneGate: 2D dHash, 4x4 ROI block max-pooling, and adaptive watchdog.
    """
    def __init__(
        self,
        alpha: float = 0.25,
        beta: float = 0.20,
        gamma: float = 0.30,
        delta: float = 0.25,
        threshold: float = 0.18,
        ambient_heartbeat: int = 60,
        active_heartbeat: int = 6,
        cooldown_interval: int = 6,
        dead_zone_eps: float = 7.0,
        hazard_vocabulary: Optional[set] = None
    ):
        tot = alpha + beta + gamma + delta
        self.alpha = alpha / tot
        self.beta = beta / tot
        self.gamma = gamma / tot
        self.delta = delta / tot
        self.threshold = threshold
        self.ambient_heartbeat = ambient_heartbeat
        self.active_heartbeat = active_heartbeat
        self.cooldown_interval = cooldown_interval
        self.dead_zone_eps = dead_zone_eps
        self.hazard_vocabulary = hazard_vocabulary or {
            'car', 'vehicle', 'truck', 'bus', 'stairs', 'curb', 'hole', 'obstacle'
        }
        self.prev_hash: Optional[np.ndarray] = None
        self.prev_emb: Optional[np.ndarray] = None
        self.prev_gray: Optional[np.ndarray] = None
        self.cached_hazard: float = 0.0
        self.frames_since_invocation: int = 0
        self.frames_since_hazard_alert: float = float('inf')
```

```
    def compute_2d_dhash(self, img: Image.Image, size: int = 16) -> np.ndarray:
        gray = img.resize((size + 1, size + 1), Image.Resampling.LANCZOS).convert('L')
        pix = np.array(gray, dtype=np.float32)
        diff_h = pix[:size, 1:size+1] - pix[:size, :size]
        diff_v = pix[1:size+1, :size] - pix[:size, :size]
        bh = diff_h > self.dead_zone_eps
        bv = diff_v > self.dead_zone_eps
        return np.concatenate([bh.flatten(), bv.flatten()])

    def compute_appearance_proxy(self, img: Image.Image) -> np.ndarray:
        arr = np.array(img.convert('RGB'))
        r, _ = np.histogram(arr[:, :, 0], bins=16, range=(0, 256), density=True)
        g, _ = np.histogram(arr[:, :, 1], bins=16, range=(0, 256), density=True)
        b, _ = np.histogram(arr[:, :, 2], bins=16, range=(0, 256), density=True)
        vec = np.concatenate([r, g, b]).astype(np.float32)
        norm = np.linalg.norm(vec)
        return vec / (norm + 1e-7)

    def compute_motion_proxy(self, cur_gray: np.ndarray, prev_gray: np.ndarray) -> Tuple[float, float, float]:
        diff = np.abs(cur_gray - prev_gray) / 255.0
        s_mot_global = float(np.mean(diff))
        block_means = diff.reshape(4, 8, 4, 8).mean(axis=(1, 3))
        s_roi_max = float(block_means.max())
        mean_b = float(block_means.mean())
        std_b = float(block_means.std())
        cv_spatial = float(std_b / (mean_b + 1e-5))
        return s_mot_global, s_roi_max, cv_spatial

    def evaluate_change(
        self,
        img: Image.Image,
        detection_labels: Optional[List[str]] = None
    ) -> Tuple[bool, float, Dict[str, Any]]:
        cur_hash = self.compute_2d_dhash(img)
        cur_emb = self.compute_appearance_proxy(img)
        cur_gray = np.array(img.resize((32, 32), Image.Resampling.BILINEAR).convert('L'), dtype=np.float32)

        if self.prev_hash is None:
            self.prev_hash = cur_hash
            self.prev_emb = cur_emb
            self.prev_gray = cur_gray
            self.frames_since_invocation = 0
            if detection_labels is not None:
                has_haz = any(l.lower() in self.hazard_vocabulary for l in detection_labels)
                self.cached_hazard = 1.0 if has_haz else 0.0
                self.frames_since_hazard_alert = 0 if has_haz else float('inf')
            breakdown = {
                's_vis': 1.0, 's_app': 1.0, 's_mot': 1.0, 's_roi_max': 1.0,
                'cv_spatial': 0.0, 's_haz': self.cached_hazard, 'score': 1.0,
                'heartbeat_forced': False
            }
            return True, 1.0, breakdown

        s_vis = float(np.mean(cur_hash != self.prev_hash))
        cos_sim = float(np.dot(cur_emb, self.prev_emb))
        s_app = float(np.clip(1.0 - cos_sim, 0.0, 1.0))
        s_mot_global, s_roi_max, cv_spatial = self.compute_motion_proxy(cur_gray, self.prev_gray)
        s_mot = float(0.50 * s_mot_global + 0.50 * s_roi_max)

        # Watchdog heartbeat interval selection
        is_active = (s_mot >= 0.02) or (s_roi_max >= 0.05) or (self.cached_hazard > 0)
        heartbeat_limit = self.active_heartbeat if is_active else self.ambient_heartbeat

        # Hazard state with cooldown and critical override
        self.frames_since_hazard_alert += 1
        override = (s_roi_max >= 0.35) or (s_mot >= 0.20)
        if self.cooldown_interval > 0 and self.cached_hazard > 0:
            s_haz = 0.0 if (self.frames_since_hazard_alert < self.cooldown_interval and not override) else
float(self.cached_hazard)
        else:
            s_haz = float(self.cached_hazard)

        change_score = float(
            self.alpha * s_vis +
            self.beta * s_app +
            self.gamma * s_mot +
            self.delta * s_haz
```

```
        )

        self.frames_since_invocation += 1
        heartbeat_forced = bool(
            heartbeat_limit is not None and
            self.frames_since_invocation >= heartbeat_limit
        )
        should_invoke = bool(change_score >= self.threshold or heartbeat_forced)

        if should_invoke:
            self.prev_hash = cur_hash
            self.prev_emb = cur_emb
            self.prev_gray = cur_gray
            self.frames_since_invocation = 0
            if detection_labels is not None:
                has_haz = any(l.lower() in self.hazard_vocabulary for l in detection_labels)
                if has_haz:
                    self.frames_since_hazard_alert = 0
                    self.cached_hazard = 1.0
                else:
                    self.cached_hazard = 0.0
                    self.frames_since_hazard_alert = float('inf')

        breakdown = {
            's_vis': s_vis, 's_app': s_app, 's_mot': s_mot,
            's_mot_global': s_mot_global, 's_roi_max': s_roi_max,
            'cv_spatial': cv_spatial, 's_haz': s_haz,
            'score': change_score, 'heartbeat_forced': heartbeat_forced
        }
        return should_invoke, change_score, breakdown
```

B. Priority-Weighted Audio Queue Reference Implementation

Below is the complete reference implementation of the PriorityAudioQueue class with preemption, ducking, and acoustic earcon generation.

```
import time
from typing import Optional, List, Dict, Any

class PriorityAudioQueue:
    """
    PriorityAudioQueue: Priority-weighted speech synthesis with <=150 ms preemption.
    """
    def __init__(self, tts_engine: Any, preemption_threshold: float = 0.70):
        self.tts = tts_engine
        self.preemption_threshold = preemption_threshold
        self.is_speaking = False
        self.current_utterance: Optional[Dict[str, Any]] = None

    def compute_priority(
        self,
        entity_label: str,
        hazard_vocab: set,
        distance_m: float,
        speed_mps: float,
        is_novel: bool
    ) -> float:
        f1 = 1.0 if entity_label.lower() in hazard_vocab else 0.0
        f2 = max(0.0, min(1.0, 1.0 - distance_m / 10.0))
        f3 = max(0.0, min(1.0, speed_mps / 5.0))
        f4 = 1.0 if is_novel else 0.0
        return float(0.45 * f1 + 0.30 * f2 + 0.15 * f3 + 0.10 * f4)

    def enqueue_utterance(self, text: str, priority_score: float, is_hazard: bool = False):
        if self.is_speaking and priority_score >= self.preemption_threshold:
            # Immediate Audio Preemption (FM-06 mitigation)
            self.tts.stop()  # Interrupts ongoing low-priority ambient speech (<15 ms)
            self.play_earcon(frequency=880, duration_ms=30)  # Warning cue
            self.tts.speak(text)  # Synthesizes urgent hazard warning
            self.current_utterance = {'text': text, 'prio': priority_score, 'hazard': True}
            self.is_speaking = True
        elif not self.is_speaking:
            self.tts.speak(text)
            self.current_utterance = {'text': text, 'prio': priority_score, 'hazard': is_hazard}
            self.is_speaking = True

    def play_earcon(self, frequency: int, duration_ms: int):
```

```
# Plays auditory icon to alert BLV user to high-priority hazard
pass
```